

# Azure AI Content Understanding SDK - guida pratica

Questa guida permette di ripetere da zero i cinque esercizi della repository usando notebook Jupyter autonomi e senza `main()`. La cartella `esercizi` contiene le tracce con celle vuote; `soluzioni` contiene i notebook completi corrispondenti. Gli esempi sono basati su `azure-ai-contentunderstanding 1.2.0b3`, versione preview.

## 1. Cosa serve

- Python 3.12 o successivo.
- Azure CLI.
- Una sottoscrizione Azure.
- Una risorsa Microsoft Foundry con Content Understanding in una regione supportata.
- Il ruolo `Cognitive Services User` sulla risorsa.
- Una risorsa già configurata per gli analyzer del laboratorio.

I notebook non gestiscono deployment e non eseguono controlli preventivi: eventuali errori vengono restituiti direttamente da Python o Azure.

## 2. Ambiente locale

```
python -m venv .env
.\.env\Scripts\python.exe -m pip install -r requirements.txt
.\.env\Scripts\python.exe -m ipykernel install --prefix .env --name cu-lab --display-name "Python (CU lab)"
Copy-Item .env.example .env
az login
.\.env\Scripts\python.exe -m jupyter lab --notebook-dir=.
```

Seleziona il kernel `Python (CU lab)` in Jupyter, oppure l'interprete del `venv` dal selettore kernel di VS Code. Usa `Restart Kernel and Run All` per provare una soluzione da zero. Le celle si eseguono in ordine: import e configurazione, creazione del client e analisi, stampa dei risultati. I notebook non scrivono file di risultato e non creano cartelle `output` o `segmenti`. Apri una traccia in `esercizi` o il notebook completo in `soluzioni`. La directory di lavoro del kernel deve essere quella del notebook. I percorsi sono fissi, senza ricerca automatica della root o del file `.env`:

```
load_dotenv(r"..\.env", override=True)
DOCUMENTO = Path(r"..file\esercizio1.pdf")
document_bytes = DOCUMENTO.read_bytes()
```

Il file `.env` contiene:

```
AZURE_CONTENTUNDERSTANDING_ENDPOINT=https://<risorsa>.services.ai.azure.com/
AZURE_CONTENTUNDERSTANDING_CUSTOM_EXPENSE_ANALYZER_ID=lab_spese_custom
AZURE_CONTENTUNDERSTANDING_CUSTOM_CLASSIFIER_ANALYZER_ID=lab_classificatore_documenti
```

Non inserire chiavi o segreti nel codice o nel controllo versione. `DefaultAzureCredential` usa l'identità Azure disponibile; in locale può usare la sessione creata da `az login`.

## 3. Creare e chiudere il client

```
credential = DefaultAzureCredential()
client = ContentUnderstandingClient(
    endpoint=os.environ["AZURE_CONTENTUNDERSTANDING_ENDPOINT"],
    credential=credential,
)
try:
    # Operazioni Content Understanding.
    ...
finally:
    client.close()
    credential.close()
```

`endpoint` è l'endpoint della risorsa. `credential` firma le richieste. La costruzione non avvia un'analisi.

## 4. Analyzer prebuilt

Un analyzer prebuilt esiste già nel servizio: il suo ID viene scritto direttamente nella chiamata SDK, senza una variabile dedicata:

```
analyzer = client.get_analyzer(analyzer_id="prebuilt-receipt.hotel")
```

``get_analyzer`` recupera schema, configurazione e ruoli modello. Non crea né modifica l'analyzer.

## 5. Analizzare un file

```
result = client.begin_analyze_binary(  
    analyzer_id="prebuilt-receipt.hotel",  
    binary_input=documento.read_bytes(),  
).result()
```

``analyzer_id`` sceglie l'analyzer. ``binary_input`` contiene i byte del file. ``begin_analyze_binary`` restituisce un poller; ``result()`` attende il completamento.

L'output più utile è:

- ``result.contents``: contenuti riconosciuti.
- ``content.markdown``: testo e layout in Markdown.
- ``content.fields``: campi strutturati.
- ``field.value``, ``field.confidence`` e ``field.source``: valore, affidabilità e origine.
- ``result.as_dict()``: oggetto serializzabile in JSON.

## 6. Analyzer custom

Un custom analyzer definisce uno schema coerente con il dominio. I metodi campo usati negli esercizi sono:

- ``extract``: estrae un valore presente nel documento.
- ``generate``: produce un valore o una sintesi usando il modello completion.
- ``classify``: assegna una voce fra quelle definite in ``enum``.

Le definizioni risiedono nella cartella ``modelli`` della root, separata da ``esercizi`` e ``soluzioni``, e sono solo JSON:

``nota_spese.json`` e ``classificatore_documenti.json``. Usano i nomi del formato API (``baseAnalyzerId``, ``fieldSchema``, ``returnDetails``). Contengono solo la definizione: nessun ``contents``, risultato estratto o risposta del servizio. ``fieldSchema`` e ``contentCategories`` descrivono cosa analizzare, non il risultato di un'analisi. Il caricamento non crea ancora un analyzer nel servizio:

```
from azure.ai.contentunderstanding.models import ContentAnalyzer  
  
# Legge la definizione JSON in UTF-8 e la converte in un dizionario.  
definizione = json.loads(  
    Path(r"..modelli\nota_spese.json").read_text(encoding="utf-8")  
)  
# Costruisce l'oggetto SDK a partire dalla definizione JSON.  
custom = ContentAnalyzer(definizione)
```

L'ID dell'istanza custom arriva da ``env``, tramite le variabili

``AZURE_CONTENTUNDERSTANDING_CUSTOM_EXPENSE_ANALYZER_ID`` e  
``AZURE_CONTENTUNDERSTANDING_CUSTOM_CLASSIFIER_ANALYZER_ID``.

I ruoli modello sono già dichiarati nei JSON: la nota spese usa completion ed embedding, il classificatore solo completion. I notebook inviano la definizione direttamente, senza leggere o verificare deployment.

Creazione e pulizia:

```
client.begin_create_analyzer(  
    analyzer_id=custom_id,  
    resource=custom,  
    allow_replace=True,  
).result()  
try:  
    result = client.begin_analyze_binary(  
        analyzer_id=custom_id,  
        binary_input=documento.read_bytes(),  
    ).result()  
finally:  
    client.delete_analyzer(analyzer_id=custom_id)
```

``resource`` è la definizione custom. ``allow_replace=True`` rende ripetibile il laboratorio quando un'esecuzione precedente ha lasciato lo stesso ID. La cancellazione va eseguita in ``finally``. Nei notebook custom ``with DefaultAzureCredential()`` e ``with ContentUnderstandingClient(...)`` chiudono automaticamente credenziale e client, anche in caso di errore.

## 7. Preparare il risultato per un LLM

```
contesto = to_llm_input(result)
solo_campi = to_llm_input(result, include_markdown=False)
solo_markdown = to_llm_input(result, include_fields=False)
```

Il metodo `to_llm_input` opera localmente. Per impostazione predefinita combina campi e Markdown. I due flag permettono di escludere una delle sezioni.

## 8. Classificazione e segmentazione

Un classificatore usa categorie con descrizioni chiare:

```
categoria = ContentCategoryDefinition(
    description="Standalone restaurant receipt for meals and drinks."
)
config = ContentAnalyzerConfig(
    return_details=True,
    enable_segment=True,
    allow_in_page_segments=False,
    content_categories={"restaurant_receipt": categoria},
)
```

Il parametro `enable_segment=True` trova più documenti nello stesso file. Il parametro `allow_in_page_segments=False` impedisce segmenti multipli nella stessa pagina. I segmenti restituiscono categoria, confidence, pagina iniziale e pagina finale.

Content Understanding restituisce gli intervalli, non nuovi PDF. Il notebook mostra soltanto il risultato, senza dividere fisicamente il documento:

```
# Mostra categoria, intervallo di pagine e confidence di ciascun segmento.
for content in result.as_dict()["contents"]:
    for segment in content["segments"]:
        print(
            segment["category"],
            segment["startPageNumber"],
            segment["endPageNumber"],
            segment["confidence"],
        )
```

Gli intervalli di CU sono inclusivi e partono da 1. Il notebook stampa le classificazioni restituite dal servizio, senza controlli aggiuntivi o letture locali delle pagine.

## 9. Errori frequenti

- `KeyError` sull'endpoint: manca la variabile nel file `.env`.
- `ClientAuthenticationError`: eseguire `az login` e verificare tenant/account.
- `HttpResponseError` con autorizzazione negata: verificare il ruolo sulla risorsa.
- Analyzer già esistente: usare un ID dedicato o `allow_replace=True`.
- Nessun segmento: migliorare le descrizioni delle categorie e verificare il PDF.

## 10. Sequenza di studio

Per ogni punto completa prima la traccia in `esercizi`, poi confrontala con il notebook omonimo in `soluzioni`.

1. Eseguire `01-client.ipynb` per verificare configurazione e costruzione client. 2. Eseguire `02-analyzer.ipynb` per osservare lo schema prebuilt. 3. Eseguire `03-analisi.ipynb` e leggere Markdown, campi, confidence e source. 4. Eseguire `04-output-llm.ipynb` per schema custom e `to_llm_input`. 5. Eseguire `05-classificazione.ipynb` per classificazione e intervalli di pagine.

Ogni notebook ricrea client e credenziale e usa percorsi relativi fissi dalla directory di lavoro del notebook (`esercizi` o `soluzioni`). Non usa `__file__`, che non esiste nelle celle Jupyter, né richiede risultati di un altro esercizio.

Le operazioni Azure rimangono nella stessa cella del client: `finally` o `with` chiudono le risorse anche se una chiamata fallisce. I custom vengono eliminati dopo l'analisi. Un riavvio forzato del kernel durante una chiamata può lasciare l'analyzer sul servizio.

## 11. Riferimenti ufficiali

- SDK Python:

<https://learn.microsoft.com/python/api/overview/azure/ai-contentunderstanding-readme>

- Custom analyzer:

<https://learn.microsoft.com/azure/ai-services/content-understanding/tutorial/create-custom-analyzer>

- Classificazione:

<https://learn.microsoft.com/azure/ai-services/content-understanding/how-to/classification-content-understanding-studio>

- Concetti sui classifier:

<https://learn.microsoft.com/azure/ai-services/content-understanding/concepts/classifier>

- Sample Python:

<https://github.com/Azure/azure-sdk-for-python/tree/main/sdk/contentunderstanding/azure-ai-contentunderstanding/samples>